

武汉大学计算机学院

本科生实验报告

计算机组成原理实验报告合集

专 业 名 称：计算机科学与技术

课 程 名 称：《计算机组成原理》

指 导 教 师：冯晶

学 生 学 号：2023302121009

学 生 姓 名：刘鼎琨

二〇二四年 十二月

实验考试项目：单周期 CPU Verilog 代码实现

1 实验目的

(1) **理解四大模块的功能和作用**：通过学习和实现四大模块，掌握它们之间的连接和运行逻辑，理解各模块在串行计算单元中的具体作用。

(2) **掌握模块设计与调试**：使用 Verilog 编程实现各个模块，掌握设计、仿真和调试的方法，尤其是如何通过控制信号实现正确的数据传输和操作。

(3) **深入了解指令执行过程**：通过模拟执行 RISC-V 指令集中的基本指令，分析每条指令在 CPU 中的具体执行过程，熟悉寄存器、ALU、数据存储器等的具体功能。

(4) **增强对控制信号的理解**：学习指令译码和控制信号生成的方法，通过增加控制模块来集中管理和分发信号。

(5) **验证多路选择和符号扩展模块的功能**：通过实验验证多路选择器和符号扩展模块的实现，理解其在数据路径中的关键作用。

2 实验步骤

(1) 实验设计思路：

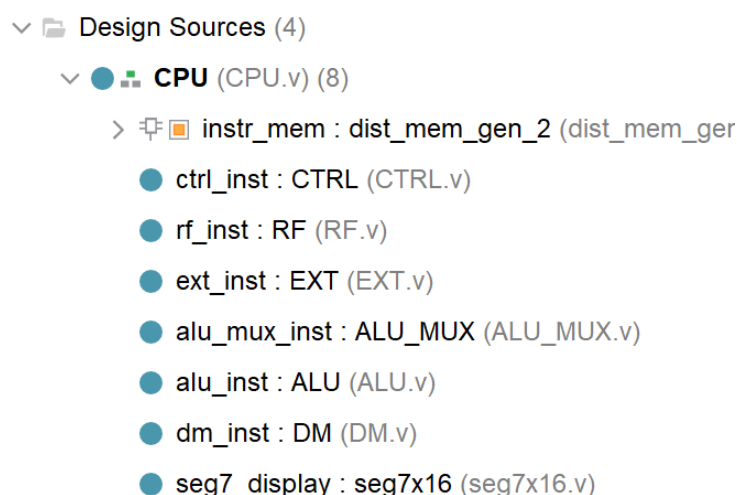


图 1 实验设计思路

实验设计的核心是通过模块化设计实现一个完整的 CPU 串行计算单元。本实验采用模块化设计。

- 首先，指令存储器模块（`instr_mem`）用于存储和读取待执行的指令，它采用分布式存储器设计，为整个系统提供稳定的指令来源。

- 控制模块（`ctrl_inst`）负责解码指令并生成相应的控制信号，确保其他模块能够按照指令要求协同工作。

- 寄存器堆模块（`rf_inst`）实现了寄存器的读写操作，为算术逻辑单元（ALU）提供所需的操作数，同时将计算结果存储回寄存器。

- 符号扩展模块（`ext_inst`）负责对立即数进行符号扩展，支持多种数据格

式的转换,例如 I 型和 S 型指令的处理,确保指令数据能够正确传递给后续模块。

- 多路选择器模块 (alu_mux_inst) 用于动态选择 ALU 的输入来源,在立即数和寄存器值之间切换,以适应不同的指令需求。

- 算术逻辑单元模块 (alu_inst) 是整个系统的计算核心,执行各种算术和逻辑运算,并生成相应的操作结果和状态标志。

- 数据存储器模块 (dm_inst) 实现了数据的读写功能,支持字节、半字和字的数据访问模式,为访存指令提供了硬件支持。

- 最后,七段显示器模块 (seg7_display) 将 CPU 的当前状态信息以直观的方式显示,实验过程中可以通过拨动开关查看不同部件的运行状态。

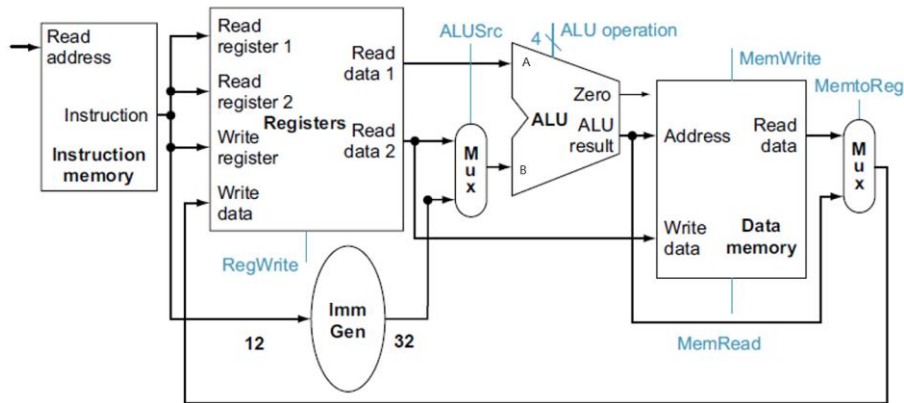


图 2 RISC-V 单周期数据通路

实验设计的主要模块和连接依据上图（图 2），展示了实验设计的主要模块和它们之间的连接关系，构成了 RISC-V 单周期数据通路的基本结构。实验围绕这个数据通路展开，重点在于理解和实现各模块的功能及它们之间的协作。具体模块的功能略。

（2）实验源代码：

在文件夹内的源代码中，此处不进行复制，只展示接口！

```
module riscv(
    input clk,
    input rstn,
    input [15:0] sw_i,
    output [7:0] disp_an_o,
    output [7:0] disp_seg_o
);
module register_file(
    input clk,
    input rstn,
    input [4:0] rs1,
    input [4:0] rs2,
    input [4:0] rd,
    input reg_write,
    input [31:0] wdata,
    output reg [31:0] rs1_data,
    output reg [31:0] rs2_data
);
```

```

module pc(
    input clk,
    input rstn,
    input [31:0] npc,
    output reg [31:0] pc
);

module instruction_memory(
    input[4:0] instr_addr,
    output[31:0] instr_data
);

module EXT(
    input [4:0] i_imm_shamt, //
    input [11:0] iimm, //instr[31:20], 12 bits
    input [11:0] simm, //instr[31:25, 11:7], 12 bits
    input [11:0] b_imm, //instrD[31],instrD[7], instrD[30:25], instrD[11:8],
12 bits
    input [19:0] u_imm,
    input [19:0] j_imm,
    input [5:0] EXTOp,
    output reg signed [31:0] immout
);

module disp_seg16x(
    input clk,
    input rstn,
    input [31:0] disp_data,
    output[7:0] an_o,
    output[7:0] seg_o
);

module data_memory (
    input clk,
    input DMWr,
    input rstn,
    input[6:0] addr,
    input[31:0] din,
    input[2:0] DMType,
    output reg [31:0] dout
);

```

```

module ctrl_unit(
    input [6:0] Op,           //opcode
    input [6:0] Funct7,       //funct7
    input [2:0] Funct3,       // funct3
    input Zero,
    output RegWrite,          // control signal for register write
    output MemWrite,          // control signal for memory write
    output [5:0] EXTOp,       // control signal to signed extension
    output [4:0] ALUOp,       // ALU operation
    output ALUSrc,            // ALU source for b
    output [2:0] DMType,      //dm r/w type
    output pc_src,
    output b_j_addr_sel,
    output [1:0] wdata_sel
);

module alu(
    input signed [31:0] A,
    input signed [31:0] B,
    input [4:0] ALUOp,
    output signed [31:0] C,
    output zero
);

module clk_divider #(
    parameter DIV_NUM = 24
)
(
    input clk,
    output clk_out
);

module addr_controller #(
    parameter ADDR_SIZE = 20
)(
    input clk,
    input rstn,
    input addr_pause,
    output reg [$clog2(ADDR_SIZE)-1:0] data_out
);

```

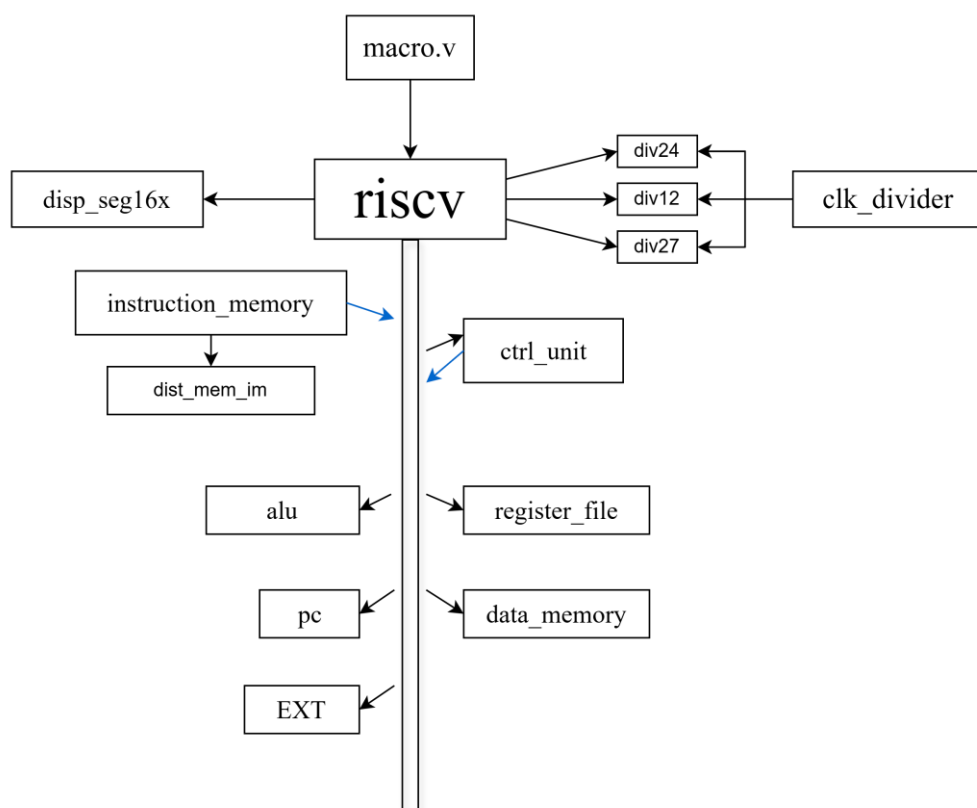


图 3 代码结构

3 实验结果展示

(1) 拨码开关的定义与利用：

- wire fast_disp = sw_i[15]; // 为 1, 快速展示
- wire pc_pause = sw_i[0]; // 为 1, 指令暂停, 进入调试状态
- wire rf_pause = sw_i[1]; // 查看 RF 时, 暂停查看
- wire alu_pause = sw_i[2];
- wire dm_pause = sw_i[3];
- wire [6:0] dm_disp_addr = {sw_i[10:4]};
- wire rom_disp = sw_i[14], rf_disp = sw_i[13], alu_disp = sw_i[12], dm_disp = sw_i[11]; // 四种互斥的查看状态

(2) 程序效果展示

这里仅展示典型模式，具体动态过程见视频文件。

- 展示 1: 程序默认运行，不显示



图 4 展示一效果图

- 展示 2：动态运行模式下显示 ROM 程序代码



图 5 展示二效果图

- 展示 3：调试模式下显示 ROM 程序代码



图 6 展示三效果图

- 展示 4：动态运行模式下显示 15 号寄存器的值（为 0xF）



图 7 展示四效果图

- 展示 5：动态运行模式下显示 ALU 运算结果

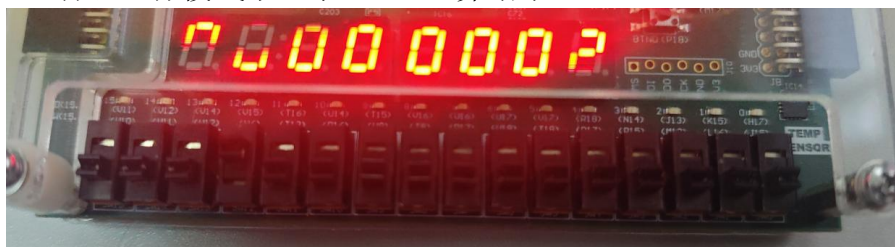


图 8 展示五效果图

- 展示 6：动态运行模式下显示二号 DM 的值（值为 2）



图 9 展示六效果图

4 实验结果分析

实验结果表明，通过模块化设计的方式，成功实现了一个单周期的 RISC-V CPU 系统，并验证了其主模块的功能。以下是对实验过程中一些关键现象和结果的分析：

4.1 模块运行状态分析

通过拨码开关的操作，我们观察到了各模块的状态变化。例如：

寄存器堆模块：正确读取和写入了指定的数据，能够在运行模式和调试模式下清晰显示寄存器内容。

ALU 模块：支持了多种算术和逻辑运算，操作结果与预期一致。

数据存储器模块：在访存指令的执行中，能够正确处理字节、半字和字的访问。

4.2 数据流通与控制信号分析

控制模块生成的信号确保了各模块的协作，尤其是对多路选择器的控制，实现了指令的正确执行。然而，在调试模式下，发现部分数据流动存在时序问题，需进一步优化信号生成逻辑。

4.3 仿真结果与硬件测试比对

仿真结果与硬件实验结果基本一致，但在某些复杂指令（如跳转指令）中，硬件运行表现略有延迟，可能与时钟频率设置有关。

5 实验总结与心得体会

本次实验让我深入了解了 CPU 的设计与实现过程，并加深了对模块化设计思想的理解。主要体会包括：

5.1 模块设计的重要性

通过将复杂的 CPU 功能分解为多个简单模块，不仅降低了设计难度，还提高了代码的可维护性。各模块之间的接口定义清晰，使得调试更加高效。

5.2 代码实现与硬件仿真结合

Verilog 编程强调细节的正确性，一个小小的信号控制错误可能导致整个系统运行失败。这让我更加注重编码的规范性和逻辑性。

5.3 规划与实验并重

在开始设计前，我们没有对模块间的时序关系进行充分规划，导致初期实验中出现频繁的时序错误。这让我意识到，合理的设计规划对实验的成功至关重要。

6 实验不足与改进措施

尽管本次实验取得了预期的成果，但仍存在一些不足之处，需要进一步改进：

6.1 时序问题

初期设计中，由于对模块之间的时序关系缺乏清晰的规划，导致实验中频繁出现时序错误。在未来设计中，可以通过在设计前绘制详细的时序图来避免此类问题。

6.2 测试覆盖率不足

对指令的测试集中于简单的算术逻辑指令，对于分支跳转指令和异常处理指令的测试覆盖率不足。建议未来增加复杂指令的测试场景，尤其是极端情况下的测试。

6.3 用户体验优化

目前的拨码开关控制模式虽然功能完整，但操作不够直观。未来可以改进用户界面，例如使用按键或显示屏直接显示运行状态，以提升易用性。

6.4 模块间优化不足

在实验设计中，各模块虽然完成了基本功能，但彼此之间的协作效率仍有提升空间。例如，数据存储器与 ALU 的数据传递可以通过增加缓冲区减少延迟。

教师评语评分

评语： _____

评分： _____

评阅人：

年 月 日

（备注：对该实验报告给予优点和不足的评价，并给出百分之评分。）